

Bit manipulation

Binary number

A **binary number** is a number expressed in the base-2 numeral system or binary numeral system, it is a method of mathematical expression which uses only two symbols: typically "0" (zero) and "1" (one).

We say that a certain bit is **set**, if it is one, and **cleared** if it is zero.

The binary number $(a_k a_{k-1} \dots a_1 a_0)_2$ represents the number:

$$(a_k a_{k-1} \dots a_1 a_0)_2 = a_k \cdot 2^k + a_{k-1} \cdot 2^{k-1} + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0.$$

For instance the binary number 1101_2 represents the number 13:

$$\begin{aligned} 1101_2 &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13 \end{aligned}$$

Computers represent integers as binary numbers. Positive integers (both signed and unsigned) are just represented with their binary digits, and negative signed numbers (which can be positive and negative) are usually represented with the [Two's complement](#).

```
unsigned int unsigned_number = 13;
assert(unsigned_number == 0b1101);

int positive_signed_number = 13;
assert(positive_signed_number == 0b1101);

int negative_signed_number = -13;
assert(negative_signed_number == 0b1111'1111'1111'1111'1111'1111'1111'0011);
```

CPUs are very fast manipulating those bits with specific operations. For some problems we can take these binary number representations to our advantage, and speed up the execution time. And for some problems (typically in combinatorics or dynamic programming) where we want to track which objects we already picked from a given set of objects, we can just use an large enough integer where each digit represents an object and depending on if we pick or drop the object we set or clear the digit.